Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

(School of Computer Engineering)

Programa de Licenciatura en Ingeniería en Computadores

(Licentiate Degree Program in Computer Engineering)

TEC

Tecnológico de Costa Rica

Generalización de framework de selección automática de pipelines de preprocesamiento a múltiples tareas de aprendizaje automático mediante meta-aprendizaje

Generalization of an Automatic Preprocessing Pipeline Selection Framework to Multiple Machine Learning Tasks through Meta-Learning

---

Informe del Anteproyecto para el Trabajo Final de Graduación

(Report of Pre-project for a Graduation Work in fulfillment of the requirements for the degree of Licentiate in Computer Engineering)

Ricardo Antonio Borbón Mena

Cartago, Junio, 2026

(Cartago, June, 2026)

Índice

Palabras Clave iv

1. Introducción 1

2. Contexto y Antecedentes 2

2.1. Descripción de la organización . . . 2

2.2. Área de conocimiento . . . 2

3. Descripción del Proyecto 3

3.1. Justificación y definición del problema . . . 3

3.1.1. Contexto del problema o necesidad . . . 3

3.1.2. Justificación de la necesidad . . . 3

3.1.3. Enunciado del problema . . . 4

3.2. Especificación de los objetivos . . . 5

3.2.1. Objetivo general . . . 5

3.2.2. Objetivos específicos . . . 5

3.3. Impactos del proyecto y resultados esperados . . . 6

3.3.1. Impactos . . . 6

3.3.2. Resultados esperados . . . 7

3.4. Alcance priorizado . . . 7

3.5. Supuestos y limitaciones . . . 8

3.5.1. Supuestos . . . 8

3.5.2. Limitaciones . . . 8

3.6. Análisis de riesgos . . . 10

4. Propuesta Metodológica 12

4.1. Tipificación del trabajo a realizar . . . 12

4.2. Descripción general del proceso por realizar 12
4.3. Criterios de éxito y diseño experimental 14
4.3.1. Corpus y presupuesto de búsqueda 14
4.3.2. Métricas primarias por tarea 15
4.3.3. Protocolo de comparación 15
4.3.4. Criterio de éxito de la hipótesis 15
4.3.5. Criterios de éxito de ingeniería 16
4.4. Tabla de relación entre objetivos y proceso por realizar 17
4.5. Cronograma propuesto 19

Referencias **21**

Apéndice. Ficha de contactos del proyecto **23**

Palabras Clave

Preprocesamiento de datos, Selección automática de pipelines (AutoML), Meta-aprendizaje (*meta-learning*), Optimización bayesiana (Optuna), Arranque en caliente (*warm-start*), Aprendizaje automático.

1 Introducción

El preprocesamiento de datos es una de las etapas que más incide en el desempeño de los modelos de aprendizaje automático (*machine learning*) [1]. La elección de las transformaciones aplicadas a los datos de entrada, su orden y sus parámetros, determina en gran medida la calidad de las predicciones, y suele exigir un proceso manual de prueba y error que consume tiempo de experimentación y conocimiento experto. La automatización de esta etapa, dentro del campo conocido como aprendizaje automático automatizado (*AutoML*), busca reducir ese esfuerzo seleccionando de forma sistemática la combinación de transformaciones que maximiza una métrica objetivo [2, 3].

En este contexto se sitúa *AutoPipeAD* [4], una librería existente en Python cuyo propósito es automatizar la selección de pipelines de preprocesamiento para la detección de anomalías en datasets de imágenes. La librería define un espacio de transformaciones componibles, genera y evalúa combinaciones mediante optimización bayesiana (Optuna) [5], ejecuta las evaluaciones en paralelo y selecciona el pipeline de mejor desempeño, generando reportes reproducibles. Está organizada en una arquitectura modular de seis componentes desacoplados (carga de datos, definición de pipelines, motor de búsqueda, evaluación, ejecución paralela y generación de reportes), y ha sido probada sobre datasets de referencia de detección de anomalías en imágenes, como MVTec AD [6] y CIFAR-10 [7] con anomalías sintéticas. Esta librería constituye el punto de partida del presente proyecto.

Sin embargo, *AutoPipeAD* presenta una limitación conceptual de alcance: aunque la motivación de una herramienta de este tipo es simplificar el preprocesamiento de datos en cualquier aplicación de aprendizaje automático, en su estado actual queda restringida a una sola tarea (detección de anomalías) y a una sola modalidad de datos (imágenes). Esta restricción responde al alcance con que fue construida y no a una limitación de su arquitectura, que está diseñada para ser modular y reemplazable componente a componente. Como se detalla más adelante, la mayor parte de sus componentes (el motor de búsqueda, la ejecución paralela, la composición de pipelines y la generación de reportes) son independientes del dominio, y el acoplamiento a imágenes y a la detección de anomalías se concentra únicamente en la carga de datos y en el cálculo de métricas.

El presente proyecto propone evolucionar *AutoPipeAD* hacia *AutoPipe*, un framework de selección automática de pipelines de preprocesamiento generalizado a múltiples tareas de aprendizaje automático, e incorporar una capa de meta-aprendizaje [8, 9] que, a partir de las características descriptivas (*meta-características*) de un dataset, recomiende configuraciones iniciales de búsqueda [10]. De esta forma se aborda tanto la generalización del

alcance como un aporte investigativo medible: determinar si el conocimiento acumulado sobre datasets previos permite acelerar la búsqueda de pipelines en datasets nuevos frente a una búsqueda sin información previa.

2 Contexto y Antecedentes

2.1 Descripción de la organización

El proyecto se desarrolla en el Instituto Tecnológico de Costa Rica (TEC), institución pública de educación superior universitaria creada mediante la Ley No. 4777 del 10 de junio de 1971, dedicada a la docencia, la investigación y la extensión en los campos de la ciencia y la tecnología. Su campus central se ubica en Cartago, Costa Rica, y cuenta con sedes y centros académicos adicionales en distintas regiones del país.

Dentro del TEC, el proyecto se enmarca en la Escuela de Ingeniería en Computadores, unidad responsable del programa de Licenciatura en Ingeniería en Computadores. Esta unidad articula la formación en las áreas de electrónica digital, arquitectura de computadores, sistemas embebidos, redes, software e inteligencia artificial. El presente Trabajo Final de Graduación (TFG) se desarrolla bajo la supervisión del profesor Luis Alberto Chavarría Zamora y toma como base una librería de software existente (*AutoPipeAD*), sobre la cual se construye la solución propuesta.

A diferencia de un proyecto realizado con una contraparte empresarial, este trabajo tiene un carácter académico e investigativo: su contraparte es la propia Área Académica de Ingeniería en Computadores, y su producto es una librería de software de uso abierto orientada a la comunidad de investigadores e ingenieros de aprendizaje automático.

2.2 Área de conocimiento

Este proyecto se ubica en el área de conocimiento de *Inteligencia Artificial y Aprendizaje Automático*, dentro del eje de Computación y Software de la carrera de Ingeniería en Computadores. El trabajo aborda el diseño y la implementación de un framework de software para la selección automática de pipelines de preprocesamiento, integrando técnicas de optimización (optimización bayesiana [11]), de meta-aprendizaje [8] y de evaluación experimental reproducible. De esta manera, el proyecto contribuye a dicha área al aportar un mecanismo, generalizable y empíricamente validado, para automatizar y acelerar una etapa crítica del flujo de trabajo de aprendizaje automático.

3 Descripción del Proyecto

3.1 Justificación y definición del problema

3.1.1 Contexto del problema o necesidad

La construcción de un modelo de aprendizaje automático exige, antes del entrenamiento, decidir cómo se preprocesan los datos: qué transformaciones se aplican (por ejemplo, normalización, escalado, codificación de variables categóricas, imputación de valores faltantes, reducción de ruido o redimensionamiento), en qué orden y con qué parámetros [1]. El espacio de combinaciones posibles es amplio, y la mejor combinación depende tanto de la tarea (clasificación, agrupamiento, detección de anomalías [12]) como de las características particulares del dataset. En la práctica, esta selección se realiza de forma manual y guiada por la experiencia, lo que la vuelve costosa en tiempo y dependiente de conocimiento experto.

La librería *AutoPipeAD* automatiza esta selección, pero únicamente para la tarea de detección de anomalías sobre datasets de imágenes. Su motor de búsqueda (Optuna), su mecanismo de paralelización (joblib), su abstracción de pipelines componibles (`BaseTransform` y `Pipeline`) y su generación de reportes reproducibles son, por diseño, independientes del dominio; sin embargo, los componentes de carga de datos y de cálculo de métricas están acoplados a imágenes y a métricas propias de la detección de anomalías. Esto impide que la herramienta cumpla su propósito original de ser de propósito general.

Por otra parte, cada vez que *AutoPipeAD* se enfrenta a un dataset nuevo, ejecuta la búsqueda de pipelines partiendo desde cero (*arranque en frío*), sin aprovechar ninguna información de búsquedas anteriores sobre datasets similares. Dado que la búsqueda automática consume tiempo de cómputo proporcional al número de combinaciones evaluadas, repetir el proceso completo para cada dataset nuevo representa una oportunidad de mejora: si fuera posible recomendar configuraciones iniciales prometedoras a partir de la experiencia acumulada, la búsqueda podría alcanzar buenos resultados con menos ensayos.

3.1.2 Justificación de la necesidad

Innovación. La generalización del framework a múltiples tareas, combinada con una capa de meta-aprendizaje que recomienda configuraciones iniciales de búsqueda a partir de meta-características del dataset, constituye un aporte que va más allá de la ingeniería incremental. Mientras que la selección automática de pipelines es un problema estudiado en

AutoML [2], la transferencia de conocimiento entre datasets para acelerar específicamente la selección de pipelines de preprocesamiento sigue siendo un área con margen de aporte [9], y permite formular una hipótesis verificable y defendible.

El proyecto se diferencia de las herramientas *AutoML* generalistas que ya emplean el arranque en caliente meta-aprendido, como *auto-sklearn* [3, 10], en tres aspectos: (1) el modelo base permanece como un componente externo configurable por el usuario, en lugar de seleccionarse dentro de un catálogo cerrado de estimadores; (2) el espacio de búsqueda se compone de transformaciones de preprocesamiento extensibles por el usuario mediante la interfaz `BaseTransform`, de modo que la capa de meta-aprendizaje se especializa exclusivamente en el preprocesamiento; y (3) se incluye la detección de anomalías, una familia de tareas que las herramientas generalistas de *AutoML* no cubren.

Evidencia del problema. El acoplamiento al dominio de imágenes y a la detección de anomalías es directamente observable en el diseño actual de la librería: su componente de carga de datos asume una estructura de directorios propia de imágenes (subdirectorios de muestras normales y anómalas) y su componente de evaluación calcula métricas específicas de detección de anomalías; ninguna otra modalidad de datos ni familia de tareas está soportada. El arranque en frío es igualmente verificable: el motor de búsqueda no consume ni produce ninguna base de conocimiento entre ejecuciones, por lo que cada nueva búsqueda parte siempre desde cero.

Relevancia estratégica y aporte de valor. Para la comunidad de investigadores e ingenieros de aprendizaje automático, una herramienta que automatice el preprocesamiento de forma agnóstica a la tarea reduce el tiempo de experimentación y la dependencia de conocimiento experto. La capa de meta-aprendizaje añade valor al reducir el costo de cómputo de la búsqueda, lo que se traduce en menor tiempo y menor consumo energético por experimento.

Aporte al área de conocimiento. El proyecto produce un artefacto de software reutilizable y un estudio experimental reproducible sobre la efectividad del arranque en caliente meta-aprendido, ambos pertinentes para el área de Inteligencia Artificial y Aprendizaje Automático.

3.1.3 Enunciado del problema

£Cómo generalizar la selección automática de pipelines de preprocesamiento a múltiples tareas de aprendizaje automático y reducir el costo de la búsqueda en datasets nuevos aprovechando, mediante meta-aprendizaje, el conocimiento acumulado sobre datasets pre-

vios?

3.2 Especificación de los objetivos

3.2.1 Objetivo general

Generalizar el framework *AutoPipeAD* de selección automática de pipelines de preprocesamiento a múltiples tareas de aprendizaje automático e incorporar una capa de meta-aprendizaje que recomiende configuraciones iniciales de búsqueda a partir de las meta-características del dataset, con el fin de reducir el costo de exploración en datasets nuevos manteniendo la reproducibilidad de los resultados.

3.2.2 Objetivos específicos

1. Refactorizar la arquitectura de *AutoPipeAD* hacia un núcleo agnóstico a la tarea con soporte para detección de anomalías y clasificación —y para agrupamiento (*clustering*) como extensión deseable, según el alcance priorizado—, e implementar el soporte para datos tabulares (cargador y transformaciones de preprocesamiento con semántica de ajuste y transformación), reutilizando y extendiendo los módulos de búsqueda, paralelización y composición de pipelines existentes.
2. Implementar un extractor de meta-características de datasets y construir una base de meta-conocimiento, ejecutando la selección automática de pipelines sobre un corpus de datasets públicos para las tareas soportadas.
3. Desarrollar un recomendador meta-aprendido que, a partir de las meta-características de un dataset nuevo, genere configuraciones iniciales (*warm-start*) que orienten el motor de búsqueda Optuna.
4. Validar experimentalmente la generalización del framework a múltiples tareas y la mejora aportada por el arranque en caliente meta-aprendido frente al arranque en frío, mediante validación cruzada por exclusión de dataset (*leave-one-dataset-out*) y la generación de reportes reproducibles.

3.3 Impactos del proyecto y resultados esperados

3.3.1 Impactos

Impacto técnico. El proyecto convierte una herramienta de un solo dominio en un framework agnóstico a la tarea, reutilizable para clasificación y detección de anomalías, y extensible al agrupamiento según el alcance priorizado. La capa de meta-aprendizaje introduce una capacidad que la herramienta no posee actualmente: aprovechar la experiencia de búsquedas anteriores para orientar las nuevas, reduciendo el número de ensayos necesarios.

Impacto académico y organizacional. El TFG aporta al Área Académica de Ingeniería en Computadores un artefacto de software de uso abierto y un estudio experimental reproducible sobre meta-aprendizaje aplicado a la selección de pipelines. El resultado es base potencial para trabajos futuros (otras modalidades de datos, técnicas generativas de aumento de datos) y material de referencia para cursos del área.

Impacto económico y ambiental. La reducción del número de ensayos de búsqueda mediante el arranque en caliente disminuye el tiempo de cómputo y, con ello, el consumo energético por experimento. Este criterio es coherente con el uso de optimización bayesiana en lugar de búsqueda exhaustiva, elegido por su mayor eficiencia, y refuerza la dimensión de sostenibilidad del proyecto.

Impacto en los stakeholders. Los investigadores e ingenieros de aprendizaje automático obtienen una herramienta que automatiza el preprocesamiento en más tareas y con menor costo de búsqueda. El estudiante consolida competencias en arquitectura de software, AutoML, meta-aprendizaje y experimentación rigurosa. La comunidad académica recibe resultados reproducibles y código abierto.

Impacto ético. El uso de datasets públicos con licencias de uso abierto, la reproducibilidad garantizada por control de semillas y la publicación del código sustentan un proceso transparente y verificable. El proyecto no procesa datos personales ni sensibles.

3.3.2 Resultados esperados

Tabla 1. Resultados esperados del proyecto.

#	Resultado	Descripción
RE1	Núcleo agnóstico a la tarea	Refactorización de *AutoPipeAD* con una interfaz de tarea y un registro de métricas que soportan detección de anomalías y clasificación; el agrupamiento se aborda como extensión deseable según el alcance priorizado (Sección 3.4).
RE2	Soporte de datos tabulares	Cargador de datasets tabulares y conjunto de transformaciones de preprocesamiento tabular (escalado, codificación, imputación, selección de características) construidas sobre la extensión de `BaseTransform` con semántica de ajuste y transformación (ajuste únicamente con el subconjunto de entrenamiento).
RE3	Extractor de meta-características	Componente que calcula un vector de meta-características descriptivas de un dataset (estadísticas, teórico-informativas y de complejidad).
RE4	Base de meta-conocimiento	Estructura de datos que asocia, para un corpus de datasets públicos, sus meta-características con los pipelines de mejor desempeño hallados por la búsqueda.
RE5	Recomendador meta-aprendido	Modelo que, dadas las meta-características de un dataset nuevo, produce configuraciones iniciales de *warm-start* para el motor Optuna.
RE6	Experimentos y reportes comparativos	Conjunto de experimentos reproducibles y reportes que comparan arranque en frío contra arranque en caliente y documentan la generalización por tarea.
RE7	Documentación y código entregado	Documentación técnica, manual de uso y código fuente del framework *AutoPipe* en repositorio.

3.4 Alcance priorizado

Para administrar el presupuesto de 336 horas con un margen de riesgo explícito, el alcance se organiza en dos niveles:

Alcance mínimo comprometido. Constituye el criterio de completitud del proyecto: (a) el núcleo agnóstico a la tarea con soporte para detección de anomalías (caso de continuidad de *AutoPipeAD*) y clasificación sobre datos tabulares; (b) el extractor de meta-características; (c) la base de meta-conocimiento sobre un corpus acotado de datasets públicos; (d) el recomendador de arranque en caliente integrado con el motor Optuna; (e) la validación experimental *leave-one-dataset-out* conforme a los criterios de éxito de la

Sección 4.3; y (f) la documentación técnica y el manual de uso.

Extensiones deseables. Se abordan únicamente si el avance verificado del alcance mínimo lo permite: (a) el soporte de agrupamiento (*clustering*), cuya evaluación no supervisada requiere un protocolo propio; (b) la ampliación del corpus de meta-conocimiento; y (c) un meta-modelo de recomendación más sofisticado que el recomendador por vecinos más cercanos.

La decisión de incorporar o descartar las extensiones deseables se toma en el punto de control de alcance de la semana 7 del cronograma (Figura 3), con base en el avance del alcance mínimo comprometido. Esta priorización es consistente con la mitigación del riesgo RI5.

3.5 Supuestos y limitaciones

3.5.1 Supuestos

- El estudiante dispone de una librería base funcional (*AutoPipeAD*) como punto de partida para la evolución.
- Existen datasets públicos tabulares etiquetados, con licencias de uso abierto (por ejemplo, OpenML y el repositorio UCI), en cantidad suficiente para construir un corpus de meta-conocimiento.
- El hardware local disponible (CPU multinúcleo y 8 GB de memoria RAM) es suficiente para ejecutar la búsqueda automática sobre datasets tabulares de tamaño pequeño a mediano.
- Las librerías de terceros utilizadas (scikit-learn, Optuna, joblib y una librería estándar de extracción de meta-características) se mantienen compatibles con Python 3.10 o superior durante el período del proyecto.
- El profesor tutor estará disponible para revisiones periódicas de avance y validación de resultados.

3.5.2 Limitaciones

- El alcance comprometido se restringe a datos tabulares y a las familias de tareas de clasificación y detección de anomalías (esta última proveniente de la librería base *AutoPipeAD*, como caso de continuidad); el agrupamiento se aborda como extensión deseable, según el alcance priorizado de la Sección 3.4. No se cubren texto, series temporales ni

grafos.

- El framework no entrena ni optimiza el modelo base del usuario, que permanece como un componente externo configurable a través de una interfaz compatible con scikit-learn. El único modelo entrenado dentro del proyecto es el meta-modelo interno del recomendador.
- El aumento inteligente de datos mediante técnicas generativas (por ejemplo, GAN) queda fuera de alcance, por las restricciones de tiempo y de hardware del proyecto; se documenta como trabajo futuro.
- La paralelización permanece limitada al hardware local; no se contempla distribución en clústeres ni en infraestructura de nube.
- El tamaño del corpus de meta-conocimiento está acotado por el presupuesto de cómputo y de tiempo (336 horas), lo que limita el alcance estadístico de las conclusiones sobre el meta-aprendizaje.
- La validación se realiza sobre datasets públicos; no se garantiza la transferencia de los resultados a datasets propietarios o a dominios sustancialmente distintos de los del corpus.

3.6 Análisis de riesgos

Tabla 2. Análisis de riesgos del proyecto.

#	Escenario de riesgo	Prob.	Impacto	Acciones de mitigación
RI1	El arranque en caliente meta-aprendido no supera de forma significativa al arranque en frío, comprometiendo la hipótesis central del proyecto.	Media	Alto	Los criterios de éxito y las líneas base quedan definidos de antemano en la Sección 4.3. Un resultado negativo riguroso sigue siendo un aporte válido y defendible; la generalización del framework se sostiene como contribución independiente. Analizar las causas (calidad de las meta-características, tamaño del corpus).
RI2	La construcción de la base de meta-conocimiento consume más tiempo de cómputo del previsto, al multiplicar datasets por ensayos de búsqueda.	Media	Alto	Acotar el corpus a entre 15 y 25 datasets pequeños a medianos, con un presupuesto fijo de ensayos por dataset (Sección 4.3). Paralelizar la generación con joblib y cachear resultados intermedios para evitar recomputar.

Continúa en la siguiente página

Tabla 2 – continuación

#	Escenario de riesgo	Prob.	Impacto	Acciones de mitigación
RI3	La refactorización del núcleo introduce regresiones que rompen el caso de detección de anomalías en imágenes ya soportado por la librería base.	Media	Medio	Ejecutar antes y después del refactor la suite de pruebas de regresión ya existente en la librería base (134 pruebas unitarias y de integración, con una cobertura del 97 %), ampliándola donde el refactor lo requiera. Aplicar partición de equivalencia sobre el motor de búsqueda para verificar su comportamiento.
RI4	La heterogeneidad de los datasets públicos (tipos mixtos, valores faltantes, desbalance) dificulta la extracción uniforme de meta-características.	Media	Medio	Utilizar una librería estándar de extracción de meta-características, normalizar el espacio de meta-características y definir criterios explícitos de inclusión y exclusión de datasets en el corpus.
RI5	El alcance planificado excede las 336 horas disponibles.	Media	Alto	Aplicar el alcance priorizado de la Sección 3.4: el agrupamiento es la primera extensión en descartarse en el punto de control de la semana 7. El aumento generativo de datos queda explícitamente como trabajo futuro. Revisar el avance contra el cronograma en cada hito.

4 Propuesta Metodológica

4.1 Tipificación del trabajo a realizar

El proyecto combina dos modalidades. Por una parte, corresponde al *desarrollo de un nuevo producto (innovación)*, en tanto evoluciona la librería *AutoPipeAD* hacia un framework agnóstico a la tarea e incorpora una capa de meta-aprendizaje que no existe en la herramienta actual. Por otra parte, incorpora una componente de *investigación teórico-práctica*, dado que formula y evalúa empíricamente una hipótesis verificable: si el arranque en caliente meta-aprendido reduce el costo de la búsqueda frente al arranque en frío. La estrategia respeta el principio de evolución sobre reinvención: se reutiliza la arquitectura modular existente y se construye sobre las decisiones de diseño consolidadas en la librería base (Optuna como motor de búsqueda, joblib para la paralelización y el modelo base externo configurable).

4.2 Descripción general del proceso por realizar

El proceso sigue una estrategia iterativa e incremental, organizada en etapas secuenciales donde cada fase construye sobre los resultados de la anterior, con validación continua a lo largo del desarrollo. La Figura 1 resume el flujo general y la Figura 2 ubica los componentes nuevos sobre la arquitectura de la librería base.



Figura 1. Flujo general de *AutoPipe* con la capa de meta-aprendizaje. Los bloques sombreados en tono claro son componentes nuevos o extendidos respecto de *AutoPipeAD*.

Componente	Estado en la evolución
`search_engine` (Optuna)	Reutilizado. Se integra el *warm-start* mediante inyección de ensayos iniciales (`enqueue_trial`).
`parallel_runner` (joblib)	Reutilizado sin cambios; se emplea además para construir el corpus en paralelo.
`pipeline` (`BaseTransform`, `Pipeline`)	Reutilizado y extendido. El contrato de transformación se amplía con semántica de ajuste y transformación (*fit/transform*): las transformaciones con estado (escalado, imputación, codificación) se ajustan únicamente con el subconjunto de entrenamiento, para evitar la fuga de información hacia la validación. Las transformaciones tabulares se añaden como nuevas subclases.
`reporter`	Reutilizado; se adaptan las métricas y gráficas por tarea.
`evaluator` (`MetricCalculator`)	Extendido a un registro de métricas por tarea.
`data_loader`	Extendido con un cargador de datos tabulares.
`meta_features`	**Nuevo**: extractor de meta-características.
`meta_recommender`	**Nuevo**: base de meta-conocimiento y recomendador de *warm-start*.

Figura 2. Mapa de reutilización y extensión sobre la arquitectura de *AutoPipeAD*.

El flujo general se compone de las siguientes etapas:

1. **Análisis y diseño de la generalización.** Se analiza la arquitectura de *AutoPipeAD* y se diseñan la interfaz de tarea agnóstica, el registro de métricas y la extensión del contrato de transformación con semántica de ajuste y transformación (*fit/transform*), necesaria para que las transformaciones con estado se ajusten únicamente con los datos de entrenamiento y se evite la fuga de información. Se define el espacio de transformaciones de preprocesamiento tabular y los criterios de inclusión de datasets en el corpus.

2. **Implementación del núcleo agnóstico y soporte tabular.** Se refactoriza el núcleo para desacoplar la búsqueda y la evaluación de la tarea concreta. Se implementan el cargador de datos tabulares y las transformaciones tabulares sobre el contrato extendido de `BaseTransform`. Se ejecuta y amplía la suite de pruebas de regresión existente de la librería base (134 pruebas), verificando que el caso de imágenes ya soportado sigue funcionando.

3. **Implementación del extractor de meta-características.** Se implementa el componente que calcula el vector de meta-características de un dataset (estadísticas, teórico-

informativas y de complejidad), utilizando una librería estándar de extracción de meta-características.

4. **Construcción de la base de meta-conocimiento.** Se selecciona el corpus de datasets públicos (entre 15 y 25, según los criterios de inclusión de la Sección 4.3) y se ejecuta la selección automática de pipelines sobre cada uno con un presupuesto fijo de ensayos, registrando sus meta-características y los pipelines de mejor desempeño. La ejecución se paraleliza con joblib y se cachean los resultados.
5. **Implementación del recomendador meta-aprendido.** Se entrena un meta-modelo que, dadas las meta-características de un dataset, recomienda configuraciones iniciales. Estas configuraciones se inyectan en el motor Optuna como ensayos iniciales de *warm-start*.
6. **Diseño experimental y validación.** Conforme al protocolo y a los criterios de éxito definidos en la Sección 4.3, se ejecutan los experimentos de validación cruzada por exclusión de dataset (*leave-one-dataset-out*) que comparan el arranque en frío contra el arranque en caliente, midiendo el número de ensayos para alcanzar un desempeño objetivo y el desempeño a presupuesto reducido.
7. **Generación de reportes y análisis de resultados.** Se generan los reportes reproducibles comparativos y se analizan los resultados de generalización por tarea y de mejora del *warm-start*.
8. **Documentación y entrega.** Se consolida la documentación técnica, el manual de uso y los criterios definidos, y se entrega el código fuente del framework *AutoPipe* en repositorio.

4.3 Criterios de éxito y diseño experimental

Para que la validación del proyecto sea verificable, los criterios de éxito se definen de antemano y quedan fijados en este anteproyecto.

4.3.1 Corpus y presupuesto de búsqueda

El corpus de meta-conocimiento se compone de entre 15 y 25 datasets públicos de clasificación tabular obtenidos de OpenML o del repositorio UCI, seleccionados con los siguientes criterios de inclusión: licencia de uso abierto, entre 500 y 50 000 instancias, no más de 100 características, ausencia de datos de imagen o texto embebidos y, en datasets multiclase, un máximo de 10 clases. La construcción de la base de meta-conocimiento ejecuta

la búsqueda con un presupuesto fijo de 50 ensayos de Optuna por dataset, con semillas controladas y resultados cacheados.

4.3.2 Métricas primarias por tarea

La métrica primaria es macro-F1 para clasificación y AUC-ROC para detección de anomalías; las demás métricas se reportan con fines descriptivos. Si el agrupamiento se incorpora como extensión, su métrica primaria será el coeficiente de silueta.

4.3.3 Protocolo de comparación

La hipótesis se evalúa mediante validación cruzada por exclusión de dataset (*leave-one-dataset-out*): para cada dataset del corpus, la base de meta-conocimiento se construye con los datasets restantes y se comparan dos condiciones sobre el dataset excluido, ambas con el mismo presupuesto de 30 ensayos y las mismas semillas: (a) **arranque en frío**, la búsqueda TPE estándar; y (b) **arranque en caliente**, la misma búsqueda con las 5 configuraciones recomendadas por el meta-recomendador inyectadas como ensayos iniciales (`enqueue_trial`). En ambas condiciones se emplea el mismo modelo base fijo, un clasificador sensible al preprocesamiento (regresión logística o k vecinos más cercanos de scikit-learn), de modo que las diferencias observadas sean atribuibles al pipeline de preprocesamiento y no al modelo.

Se miden dos variables por condición: (1) *ensayos hasta el objetivo*: número de ensayos necesarios para alcanzar al menos el 95 % del mejor valor de métrica hallado por el arranque en frío con el presupuesto completo; y (2) *desempeño a presupuesto reducido*: mejor valor de métrica alcanzado tras la mitad del presupuesto (15 ensayos).

4.3.4 Criterio de éxito de la hipótesis

La hipótesis se considera confirmada si el arranque en caliente reduce la mediana de *ensayos hasta el objetivo* en al menos un 30 % respecto del arranque en frío, con significancia estadística según la prueba de rangos con signo de Wilcoxon pareada por dataset ($\alpha = 0{,}05$). Si el criterio no se alcanza, el análisis riguroso de las causas (calidad de las meta-características, tamaño del corpus, diversidad de los pipelines ganadores) y la generalización del framework constituyen por sí mismos resultados defendibles, conforme a la mitigación del riesgo RI1.

4.3.5 Criterios de éxito de ingeniería

Con independencia del resultado de la hipótesis, el proyecto debe cumplir: (a) la suite de regresión de la librería base (134 pruebas) se mantiene en verde tras la refactorización; (b) los módulos nuevos alcanzan una cobertura de pruebas de al menos el 70 %; (c) los resultados son reproducibles (la misma semilla produce el mismo pipeline ganador); y (d) el flujo completo para el usuario final se mantiene expresable en no más de 20 líneas de código.

4.4 Tabla de relación entre objetivos y proceso por realizar

Tabla 3. Relación entre objetivos específicos y proceso por realizar.

Objetivo específico	**Principales actividades**	**Entregables asociados**	**Técnicas o herramientas**	**Estrategias de verificación y validación**
OE1. Refactorizar el núcleo agnóstico a la tarea e implementar el soporte tabular.	1. Diseño de la interfaz de tarea, del registro de métricas y de la extensión *fit/transform* del contrato de transformación. 2. Refactor del motor de búsqueda y la evaluación para desacoplar la tarea. 3. Implementación del cargador tabular y de las transformaciones tabulares sobre el contrato extendido de `BaseTransform`. 4. Ejecución y ampliación de la suite de pruebas de regresión existente del caso de imágenes.	1. Núcleo agnóstico a la tarea (RE1). 2. Soporte de datos tabulares (RE2).	Python, scikit-learn, Optuna, joblib, pytest, patrón Strategy, ruff y radon.	1. Pruebas de regresión que verifican que el caso de imágenes existente sigue funcionando. 2. Partición de equivalencia sobre el motor de búsqueda.

Continúa en la siguiente página

Tabla 3 – continuación

Objetivo específico	**Principales actividades**	**Entregables asociados**	**Técnicas o herramientas**	**Estrategias de verificación y validación**
OE2. Implementar el extractor de meta-características y construir la base de meta-conocimiento.	1. Implementación del extractor de meta-características. 2. Selección del corpus de datasets públicos según criterios de inclusión. 3. Ejecución paralela de la búsqueda sobre el corpus y registro de resultados.	1. Extractor de meta-características (RE3). 2. Base de meta-conocimiento (RE4).	Python, librería de meta-características, OpenML / UCI, joblib, Optuna.	1. Verificación de la consistencia del vector de meta-características sobre datasets de control. 2. Verificación de la completitud y reproducibilidad de la base de meta-conocimiento.
OE3. Desarrollar el recomendador meta-aprendido de *warm-start*.	1. Entrenamiento del meta-modelo sobre la base de meta-conocimiento. 2. Generación de configuraciones iniciales a partir de las meta-características. 3. Integración del *warm-start* con Optuna (`enqueue_trial`).	1. Recomendador meta-aprendido (RE5).	Python, scikit-learn, Optuna (`enqueue_trial`)	1. Verificación de que las configuraciones recomendadas son válidas y ejecutables. 2. Evaluación preliminar del recomendador sobre un subconjunto de datasets retenidos.

Continúa en la siguiente página

Tabla 3 – continuación

Objetivo específico	Principales actividades	Entregables asociados	Técnicas o herramientas	Estrategias de verificación y validación
OE4. Validar la generalización y la mejora del *warm-start*.	1. Instrumentación del protocolo *leave-one-dataset-out* y de las métricas de éxito definidas en la Sección 4.3. 2. Ejecución de los experimentos arranque en frío vs. arranque en caliente. 3. Generación de reportes reproducibles y análisis de resultados.	1. Experimentos y reportes comparativos (RE6). 2. Documentación y código entregado (RE7).	Python, validación cruzada *leave-one-dataset-out*, métricas por tarea, reportes PDF/HTML.	1. Comparación estadística entre arranque en frío y en caliente según los criterios de la Sección 4.3 (prueba de Wilcoxon; ensayos hasta el objetivo y desempeño a presupuesto reducido). 2. Verificación de generalización por tarea. 3. Revisión de resultados con el tutor.

4.5 Cronograma propuesto

El proyecto se desarrolla durante 12 semanas con una dedicación de 28 horas semanales, para un total de 336 horas. La Figura 3 presenta la distribución temporal de las etapas. La documentación técnica y el informe se elaboran de forma continua a partir de la semana 3, en lugar de concentrarse al cierre, y el punto de control de alcance de la semana 7 decide la incorporación o el descarte de las extensiones deseables (Sección 3.4). Las fechas exactas se ajustarán según la fecha oficial de inicio.

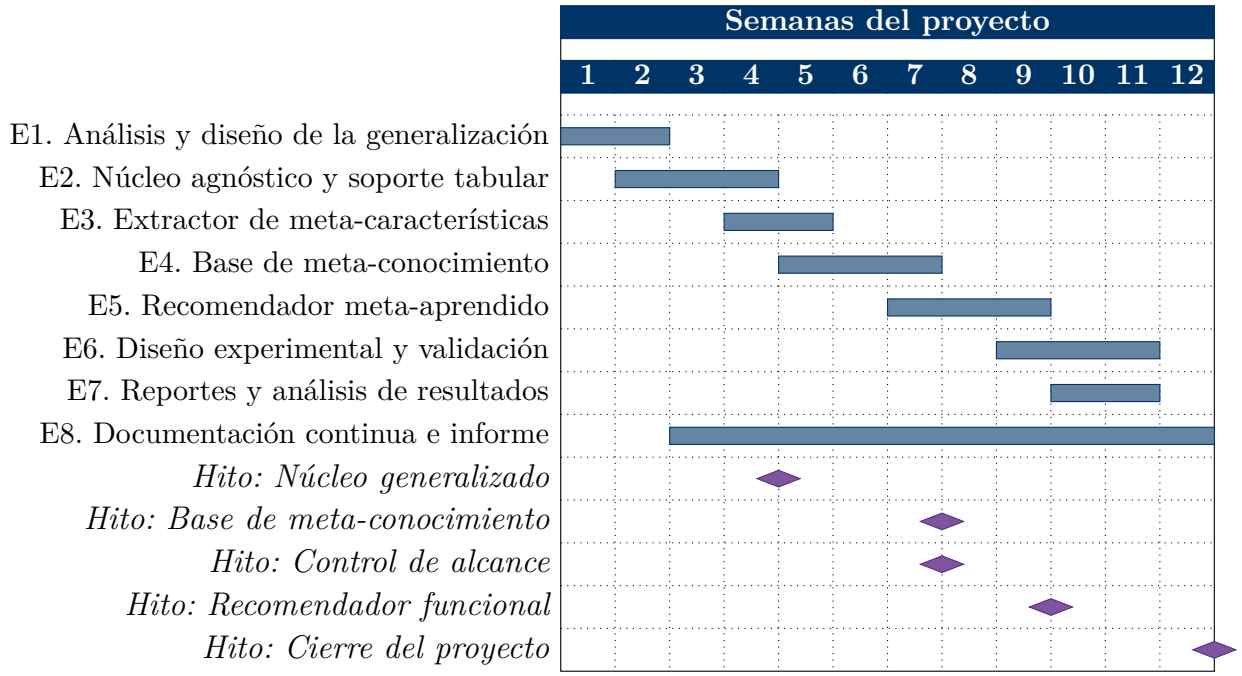


Figura 3. Cronograma general del proyecto (12 semanas, 28 horas semanales, 336 horas totales).

Referencias

[1] S. García, J. Luengo, y F. Herrera, *Data Preprocessing in Data Mining*. Cham, Suiza: Springer, 2015.

[2] F. Hutter, L. Kotthoff, y J. Vanschoren, Eds., *Automated Machine Learning: Methods, Systems, Challenges*. Cham, Suiza: Springer, 2019.

[3] M. Feurer, A. Klein, K. Eggensperger, J. T. Springenberg, M. Blum, y F. Hutter, "Efficient and robust automated machine learning," en *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015, pp. 2962–2970.

[4] R. A. Borbón Mena, "AutoPipeAD: Selección automática de pipelines de preprocesamiento para detección de anomalías en imágenes," Proyecto de Aplicación de la Ingeniería en Computadores, Instituto Tecnológico de Costa Rica, Cartago, Costa Rica, 2026. [En línea]. Disponible: `https://github.com/ItsRichard7/AutoPipeAD`

[5] T. Akiba, S. Sano, T. Yanase, T. Ohta, y M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," en *Proc. 25th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining (KDD)*, 2019, pp. 2623–2631.

[6] P. Bergmann, M. Fauser, D. Sattlegger, y C. Steger, "MVTec AD — A comprehensive real-world dataset for unsupervised anomaly detection," en *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 9592–9600.

[7] A. Krizhevsky, "Learning multiple layers of features from tiny images," Univ. de Toronto, Toronto, ON, Canadá, Reporte Técnico, 2009.

[8] J. Vanschoren, "Meta-learning: A survey," *arXiv:1810.03548*, 2018.

[9] P. Brazdil, C. Giraud-Carrier, C. Soares, y R. Vilalta, *Metalearning: Applications to Data Mining*. Berlín, Alemania: Springer, 2009.

[10] M. Feurer, J. T. Springenberg, y F. Hutter, "Initializing Bayesian hyperparameter optimization via meta-learning," en *Proc. 29th AAAI Conf. Artificial Intelligence (AAAI)*, 2015, pp. 1128–1135.

[11] J. Snoek, H. Larochelle, y R. P. Adams, "Practical Bayesian optimization of machine learning algorithms," en *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 25, 2012, pp. 2951–2959.

[12] V. Chandola, A. Banerjee, y V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, 2009.

Apéndice. Ficha de contactos del proyecto

Ingeniería en Computadores

Ficha de contactos del proyecto

Datos del estudiante	
Nombre	Ricardo Antonio Borbón Mena
Correo electrónico	ricardobm@estudiantec.cr
Teléfono	6143-9499

Datos del proyecto	
Nombre	AutoPipe
Breve descripción	Framework en Python que generaliza la selección automática de pipelines de preprocesamiento a múltiples tareas de aprendizaje automático e incorpora una capa de meta-aprendizaje que recomienda configuraciones iniciales de búsqueda a partir de las meta-características del dataset.
Fecha de inicio	Por definir

Datos de la organización	
Nombre	Instituto Tecnológico de Costa Rica, Área Académica de Ingeniería en Computadores
Nombre del contacto	Luis Alberto Chavarría Zamora (profesor tutor)
Correo electrónico	lachavarria@itcr.ac.cr
Teléfono	2550-2565